

Phase 2 Completion Summary: Question Bank (Manual Creation)

This document summarizes the work completed by the developer (you) and the AI (me) to fully implement Phase 2 of the Alta coding project.

1. Backend Implementation

Data Models

- **Question:** Added a new Mongoose model to store coding questions, containing title, description, difficulty, status (draft/published), and createdBy (faculty).
- **QuestionVersion:** Added a version snapshot model for questions. This ensures immutability so that students' submissions are tied to the exact question description and test cases present at the time they took the test.
- **TestCase:** Added a model to store test cases (input, expectedOutput, visibility, question reference).

Controllers and Routes

- **Question Management (questionController.js, questionRoutes.js):**
 - Created endpoints for CRUD operations.
 - Implemented /publish and /unpublish endpoints to transition question states.
- **Test Case Management (testCaseController.js, testCaseRoutes.js):**
 - Created endpoints for managing test cases per question.
 - Implemented access control filters: Faculty receive all test cases (both "sample" and "hidden"), while Students receive exclusively "sample" test cases.

Tests

- **Snapshot Logic (question.test.js):** Verified that when a faculty user triggers /publish on a question, the server successfully generates a QuestionVersion snapshot.
- **Visibility Enforcements (testCase.test.js):** Verified that retrieving test cases as a faculty user returns all cases, while as a student user explicitly hides the hidden ones.
- **Bug Fixes:** Identified and fixed failing tests in organization.test.js where the mock data missed createdBy validations.

2. Frontend Implementation

UI Components

- **QuestionBank (QuestionBank.jsx):** The main library view for accessing questions.
- **Faculty Dashboard / Questions (FacultyQuestions.jsx):** A UI listing all questions created by a faculty member, showing their status (draft/published) and containing action buttons.
- **Edit Question (EditQuestion.jsx):** A form to edit draft questions (title, description, and difficulty).
- **Manage Test Cases (ManageTestCases.jsx):** An interface to create and manage test cases associated with a specific question, including a dropdown to set them as 'sample' or 'hidden'.

Routing

- **AppRoutes.jsx**: Integrated and mapped all the new React components. Wrapped the new paths (/faculty/questions, /faculty/questions/:questionId/edit, /faculty/questions/:questionId/test-cases) with RoleRoute to ensure only faculty users can access them.

Conclusion

The backend is now equipped to safely and immutably manage questions and their associated test cases, strictly controlling visibility based on user roles. The frontend provides a seamless experience for faculty to build out the Question Bank manually. A pull request has been raised successfully!